

Documentación del Proyecto Compilador

Etapas: Scanner, Parser, Tests y Comandos

1 División del Trabajo

El desarrollo de esta etapa del compilador (Scanner, Parser, Tests y comandos) fue realizado de manera colaborativa. Las responsabilidades se distribuyeron de la siguiente manera:

- **Identificación de símbolos y elaboración del `scanner.lex`:** Francisco Andreani.
- **Refactorización de la gramática:** Gribaudo Re Francisco, Francisco Andreani.
- **Elaboración del `parser.y` (Gramática en Bison):** Gribaudo Re Francisco, Valentín Pastre Thuer.
- **Unidades de test:** Valentín Pastre Thuer, Gribaudo Re Francisco, Francisco Andreani.
- **Interfaz de Línea de Comandos (CLI):** Valentín Pastre Thuer.

2 Decisiones de Diseño, Suposiciones y Aclaraciones

La decisión de diseño más importante de esta etapa fue diferenciar claramente las responsabilidades entre el análisis léxico (Scanner) y el análisis sintáctico (Parser).

- Se decidió consumir en el scanner **únicamente los símbolos terminales**.
- El scanner (`scanner.lex`) no agrupa caracteres en literales complejos o identificadores completos mediante expresiones regulares, sino que se limita a retornar tokens básicos como `DIGIT` (para números del 0 al 9) y `ALPHA` (para letras minúsculas y mayúsculas).
- Se asumió que la lógica de agrupación y repetición (las expresiones regulares de la gramática original) debía trasladarse hacia nuevas reglas dentro de la gramática del parser.
- Los operadores de múltiples caracteres, como `&&`, `||`, y palabras reservadas, son detectados como tokens únicos en el scanner (`AND_OP`, `OR_OP`, `IF`, etc.) para simplificar el árbol sintáctico.

3 Diseño y Decisiones Clave: Modificaciones a la Gramática

La gramática original propuesta utilizaba Notación EBNF, incluyendo construcciones como `[ x ]` para elementos opcionales, `x*` para cero o más repeticiones, y `x+`, para listas separadas por comas. Dado que Bison (Yacc) trabaja con gramáticas libres de contexto, fue necesario transformar todas estas expresiones eliminando la notación extendida y reemplazándola por recursividad explícita.

Las alternativas analizadas y las decisiones tomadas fueron:

- **Implementación de clausuras de Kleene (*) y repeticiones (+):** En lugar de resolver las repeticiones en el scanner, se crearon reglas recursivas (recursividad por derecha) con producciones *prime* o derivadas. Por ejemplo, la gramática original dictaba la formación de literales enteros (`int_literal --> digit digit*`). Esto se transformó en dos reglas sintácticas: `int_literal : DIGIT int_literal_prime ;` y `int_literal_prime : int_literal | /* lambda */ ;`.
- **Listas separadas por comas (x⁺ ,):** Para la declaración de variables de la forma `<type> {<id>}+ , ;`, se dividió la lógica en la regla principal `var_decl : TYPE id id_prime SEMICOLON`. El no-terminal `id_prime` se encarga de consumir recursivamente la coma y los identificadores subsecuentes (`id_prime : COMMA id id_prime | /* lambda */ ;`).
- **Elementos Opcionales ([]):** Para instrucciones como el `return`, que en la especificación original admitía opcionalmente una expresión (`return [<expr>] ;`), se optó por ramificar la regla en dos alternativas explícitas: `RETURN SEMICOLON` y `RETURN expr SEMICOLON`. Lo mismo se aplicó para la estructura *if-else*, dividiendo la regla mediante un no-terminal `else` que deriva en `ELSE block` o en vacío.

4 Detalles de Implementación Interesantes

- **Delegación Léxica al Parser:** A diferencia de las implementaciones tradicionales donde el Scanner valida la estructura completa de un número flotante o un identificador, aquí el Parser (`parser.y`) asume esa responsabilidad. Por ejemplo, un número de punto flotante se construye sintácticamente uniendo literales enteros separados por un token de punto: `float_literal: int_literal DOT int_literal`. De igual forma, los identificadores alfanuméricos se arman a través de recursividad: `id : ALPHA alpha_num_prime ;` donde `alpha_num_prime` permite concatenar más letras, dígitos o guiones bajos.
- **Manejo de Precedencia y Asociatividad:** Para evitar conflictos de desplazamiento/reducción (*shift/reduce*) típicos en las gramáticas de expresiones aritméticas y lógicas, se declararon directivas de precedencia por izquierda (`%left`) en Bison. Se estableció que operadores lógicos como `AND_OP` y `OR_OP` tienen una jerarquía, seguidos por operadores aritméticos de suma/resta (`ADD_OP`, `SUBTRACT_OP`) y finalmente multiplicación/división (`MULT_OP`, `DIV_OP`, `MOD_OP`).
- **Archivos Auxiliares (CLI):** El proyecto incluye una estructura de línea de comandos que permite controlar hasta qué etapa compilar (`-target scan`, `-target parse`, etc.) y banderas de *debug* que facilitarán la visualización de la ejecución en las siguientes etapas del proyecto.
- **Integración de Ceedling:** Para el testeo de unidad utilizamos Ceedling, el cual es un sistema de construcción (*build system*) centrado en pruebas para C. Esta herramienta automatiza la compilación y ejecución de las pruebas integrando *frameworks* como Unity y CMock. Como se refleja en nuestro archivo de configuración `project.yml`, el entorno está estructurado para separar el código fuente (`src/**` y `header/**`) de los archivos de testeo (`test/**`). Adicionalmente, habilitamos el uso de plugins como `module_generator`, que nos permite crear rápidamente plantillas de archivos y tests, y `report_tests_pretty_stdout`, para obtener una salida de consola más limpia y legible al momento de evaluar los resultados de los casos de prueba.

5 Instrucciones de Compilación y Ejecución

Para facilitar la revisión y prueba del proyecto, a continuación se detallan los comandos necesarios para compilar el código fuente, ejecutar las pruebas y construir la versión final.

1. Instalación de Dependencias (Ceedling)

Para la ejecución automatizada de los tests y la construcción del proyecto es necesario contar con Ceedling. Dado que esta herramienta está construida sobre Ruby, se instala de manera global a través de su gestor de paquetes (*gem*):

```
gem install ceedling --no-document
```

2. Compilación Manual (Flex, Bison y GCC)

Si se desea generar los analizadores léxico y sintáctico de forma manual, y luego compilar el ejecutable del proyecto, se deben ejecutar los siguientes comandos:

- **Generar el Scanner (Flex):**

```
flex --header-file=header/scanner.h -o src/lex.yy.c src/scanner.lex
```

- **Generar el Parser (Bison):**

```
bison src/parser.y -o src/parser.tab.c --defines=header/parser.tab.h
```

- **Compilar con GCC:**

```
gcc @compile_flags.txt src/parser.tab.c src/lex.yy.c -lfl -o compilador
```

2.5. Ejecución Rápida

Para realizar una prueba rápida redirigiendo un archivo de texto como entrada estándar al ejecutable generado:

```
./compilador < Test.txt
```

3. Testing y Construcción con Ceedling

Al estar integrado con Ceedling, la compilación y ejecución de las pruebas se puede realizar directamente desde la terminal posicionándose en la raíz del proyecto.

- **Correr tests del Parser:** `./ceedling test:parser`
- **Correr tests del Scanner:** `./ceedling test:scanner`
- **Correr todos los tests:** `./ceedling test:all`

Para compilar todo el código y obtener el ejecutable final de lanzamiento (Release), se utiliza:

```
./ceedling release
```

Nota: Este comando compila el código y genera automáticamente el ejecutable final en la ruta `build/artifacts/release`.

4. Automatización mediante Scripts (.sh)

Para facilitar el uso, se han hecho dos *scripts* de consola que automatizan la regeneración de los analizadores léxico y sintáctico (Flex y Bison) combinándolos con la construcción o evaluación en Ceedling.

Los archivos preparados para su ejecución rápida son `compile.sh` y `run-tests.sh`:

- **Para compilar todo (./compile.sh):** Regenera el Scanner y el Parser, y si esto es exitoso, ejecuta la compilación final para *release*. Su contenido interno encadena las herramientas:

```
flex --header-file=header/scanner.h -o src/lex.yy.c src/scanner.lex && \
bison src/parser.y -o src/parser.tab.c --defines=header/parser.tab.h && \
ceedling release
```

Ejecución: `./compile.sh`

- **Para correr las pruebas (./run-tests.sh):** Regenera el Scanner y el Parser, para inmediatamente después ejecutar la batería completa de pruebas unitarias. Su contenido interno es:

```
flex --header-file=header/scanner.h -o src/lex.yy.c src/scanner.lex && \
bison src/parser.y -o src/parser.tab.c --defines=header/parser.tab.h && \
ceedling test:all
```

Ejecución: `./run-tests.sh`

6 Problemas Conocidos y Testing

- **Conflictos de Shift/Reduce:** Durante la implementación del analizador sintáctico (Parser), el principal problema detectado fue la aparición de múltiples conflictos de *shift/reduce* reportados por Bison.
- **Reglas Afectadas:** Estos problemas de ambigüedad se presentaron al evaluar las expresiones con operadores, pero fundamentalmente afectaron a las reglas estructurales de la gramática: `<program>`, `<var_decl>` y `<method_decl>`.
- **Refactorización y Solución:** Para solucionar estas colisiones, tuvimos que refactorizar la gramática. Esto se logró reescribiendo las producciones de las declaraciones de variables y métodos para eliminar las ambigüedades lógicas, asegurando que el parser pudiera decidir correctamente si desplazar (*shift*) o reducir (*reduce*). Además, como se mencionó en las decisiones de diseño, las ambigüedades de los operadores se resolvieron estableciendo sus precedencias de forma explícita.
- **Estrategia de Testing:** La detección de estos problemas y la validación de las refactorizaciones se realizaron mediante la elaboración y ejecución de casos de prueba. El uso del *framework* de testing (Ceedling) nos permitió correr las pruebas tras cada cambio en la gramática, asegurando que las modificaciones para resolver los conflictos en `<program>` y las declaraciones no rompieran el reconocimiento léxico ni la estructura del resto del compilador.